Document number:	P3859R0
Date:	2025-10-05
Audience:	EWG, CWG
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Assertions are not necessarily for changing program behavior

We state that one of the primary motivations for adding contract assertions to C++ is not about generating code to be executed: it is to provide information, relevant for determining the program correctness, for tools other than compilers. In the context of this motivation, we respond to portions of the criticism in [P3394R1] and [P3835R0].

The motivation

The motivation for allowing assertions in function declarations comes primarily from the desire to support tools like static analyzers and IDEs. If the motivation was only the run-time checks, we might just as well use macros inside function bodies, as in [N4075].

The idea here is that when a function is called in the caller, the precondition and postcondition assertions become visible in the caller also, and this additional information can be used in different ways by different tools. We will demonstrate it by an example.

```
bool in_range(int lo, int hi, int val)
  pre (lo <= hi);
```

This carries a *new* piece of information: whenever this function is called, if the first argument passed to the function is greater than the second argument, we can be *sure* that the program works against the specification: it has a bug. Here "sure" means that a static analyzer can issue a high-quality warning without risking false positives. This information is "new" because the analyzer, even using a hypothetical whole program analysis, couldn't have ever figured out this condition, as there is no undefined behavior inside the implementation. The implementation is likely:

```
return lo <= val && val <= hi;
```

And without additional information injected by the precondition assertion, it would have been considered to have a *wide contact* by the analyzer.

Now, suppose this function is called in the following context:

```
if (a < 0)
  println("negative case");

return in_range(0, a, val);
```

A static analyzer can report a potential bug: either you have a redundant check or you are calling the function out of contract.

Next, consider the following usage:

```
std::cin >> min >> max;
return in_range(min, max, val);
```

A static analyzer can report a potential bug: you are passing unsanitized input to a narrow-contract function.

Next, consider the following code snippet.

```
int min = config.read("min");
int max = config.read("max");
return is_in_range(val, min, max);
```

And imagine a future IDE that recognizes contract assertions, and upon each function call it takes every precondition assertion, substitutes function argument names from the function call for the function parameter names, and displays the new expression as a hint to the programmer:

```
int min = config.read("min");
int max = config.read("max");
return is_in_range(val, min, max);
                   PRE: val <= min
```

In the above case the programmer should be alerted: why does the precondition require that `val <= min` given that the whole point is to check if `val` is within range? The user is now compelled to check the function declaration and learns that they passed the arguments in the wrong order. Bug prevented! And we didn't even employ any static analyzer.

The reason we do not have a lot of tools like this already is that we do not have a standardized notation for precondition and postcondition assertions. Note that we do not need any notion of "evaluation semantics" or "violation handlers" for the above use cases to work. Even a stripped variant of [P2900R14] that allowed only one evaluation semantic — *ignore* — would be enough to help detect and prevent bugs. Code sanitizers, akin to UB sanitizes, would still offer to inject runtime checks based on those assertions, even if such behavior were non-conformant. Note that nobody complains that UB-sanitizers "make C++ less safe", even though their run-time checks are not guaranteed to be executed.

Discussion

Controlling which assertions are run-time evaluated

We have seen a number of concerns that boil down to expressing dissatisfaction with how one can control which assertions are run-time evaluated, and if this control could reliably work. For instance, [P3835R0] observes that in the [CD] there is no way to specify the evaluation semantic for contract assertions in *all* functions from a given *library* or *software component*.

First, in should be noted that this problem can only be experienced by those who want to benefit from the run-time checking aspect of contract assertions. In contrast, those who want to benefit from tool-assisted bug detection and static analysis aspect of contract assertions are not affected by the insufficient control of run-time checks in compilers. Contract assertions offer value from day one, and have the potential to offer more value with subsequent releases.

Second, ultimately what users get is what the tool and compiler vendors have implemented. The behavior of the compiler is a contract between compiler users and compiler vendors. The C++ Standard plays a role in this contract, but this role is not omnipotent. The C++ Standard can only put limits on what a compiler can do to still be able to be called standard-conformant. Nothing more. For instance, we do not standardize the compiler's

interface, or its switches. For another instance, the Standard never specifies that the compiled code needs to be efficient, it just makes sure that what is standardized does not prevent compilers from generating efficient code.

This is also the case for controlling the run-time evaluation of contract assertions. We decided that this will be configurable outside of the source code. That is, the same source code can result in different programs with different behavior based on compiler switches. One can dispute this decision, but unless this decision is explicitly disputed, the only thing that remains for the C++ Standard to do is to make sure that it does *not prevent* any practical and useful configuration of the run-time assertion evaluation if a compiler — or generally, implementation — chooses to offer it. This has been done. Any conceivable fine-grained control over which assertion to run-time evaluate will be based on a combination of the following:

1. Classification that is based on the assertion's context and surroundings: enclosing namespace, owning module, translation unit, assertion kind.
2. Programmer's input via additional tagging of individual assertions or functions in source code,
3. Or providing an external configuration file.

#1 and #3 above to work require nothing and cannot require nothing from the C++ Standard. #2 requires the ability to add tags to contract assertions and functions, and the [CD] already offers this: attributes. Tool vendors get all they could possibly need. Any further complaints can only be addressed to vendors.

Third, a lot of concerns about some checks being elided against the product owner's intentions can be addressed by configuring the compiler to generate run-time checks from *every* compiled contract assertions and linking/importing only libraries that have been confirmed to be compiled in the mode that checks every compiled contract assertion. You can simply ban or discourage any other configuration in your environment, if you are in control and you deem it important.

Decomposition to smaller features

Finally, we need to address the suggestion from [P3829R0] that contract assertions should be composed of smaller, independent language features: function decorators, lazy evaluation and ODR controlling tools. Then anyone could just compose contract assertions of their liking from such building blocks.

Such decomposition would be detrimental to the static analysis support. Assertions being an atomic, lowest-level building block are crucial part of the design: all tools in the world need to recognize that it is a part of program specification that is being *asserted* here. We need an explicit marker to indicate exactly this: input to static analyzers and similar tools. We would not be able to get that if a runtime-checking effect is achieved by composing `if`-statements, manual calls to `std::abort()`, or the usage of any other unrelated small language feature. The main value of contract assertions is for conveying the intention to other tools and humans, not for controlling the run-time behavior.

Conclusion

Because the purpose of the C++ Standard is to specify how the source code is mapped onto the observable behavior of the compiled program, it is the behavior of the program that is specified in [P2900R14]. And one might draw a conclusion that this is underspecified. But this view fails to appreciate that the purpose of contract assertions extends beyond how programs — possibly incorrect — behave. These assertions will be consumed by tools that remain beyond the scope of the C++ Standard.

It looks like the attribute-like syntax from [P0542R5] conveyed this idea better. Interestingly, there were fewer objections on the run-time "unpredictability" grounds to [P0542R5], even though it had more such issues.

References

- [CD] — Jonathan Caves, Daniel Kruegler, Nina Ranns, Tim Song, "ISO/IEC CD 14882, Programming languages — C++", (https://www.iso.org/standard/91179.html).
- [N4075] — John Lakos, Alexei Zakharov, Alexander Beels, "Centralized Defensive-Programming Support for Narrow Contracts (Revision 6)", (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4075.pdf).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++", (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html).
- [P2900R14] — Joshua Berne, Timur Doumler, Andrzej Krzemieński et al., "Contracts for C++", (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2900r14.pdf).
- [P3376R0] — Andrzej Krzemieński, "Contract assertions versus static analysis and ‘safety’", (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3376r0.html).
- [P3829R0] — Chisnall, Spicer, Voutilainen, Dos Reis, Garcia, "Contracts do not belong in the language", (https://isocpp.org/files/papers/P3829R0.pdf).
- [P3835R0] — John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, "Contracts make C++ less safe -- full stop!", (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3835r0.html).